

DESIGN PATTERNS & CLEAN CODE

Écrire un code lisible, maintenable, évolutif
et réutilisable

ICE BREAKER



OBJECTIFS



B2 : DESIGN PATTERNS & CLEAN CODE

- Appliquer les principes du Clean Code pour rendre le code plus lisible, explicite et facile à maintenir
- Comprendre et appliquer les principes SOLID pour construire un code modulaire, évolutif et testable
- Découvrir les principales familles de Design Patterns GoF et savoir reconnaître les situations où ils sont pertinents
- Mettre en pratique Strategy, Factory, Adapter, Observer, Decorator et d'autres patterns sur des cas concrets
- Pratiquer les Object Calisthenics comme contraintes de conception pour réduire la complexité et améliorer la clarté du code
- Évaluer rapidement la qualité d'un code existant et proposer des refactorings pertinents

DÉROULÉ DES 4 JOURS

JOUR 1	JOUR 2	JOUR 3	JOUR 4
Clean Code Diagnostic, naming, code smells, refactoring	SOLID Responsabilités, abstractions, dépendances	Design Patterns Strategy, Factory, Adapter, Observer	Challenge final Audit d'un code existant, refactoring
Pratique Exercices guidés et premier nettoyage PHP	Pratique Refactoring, découpage, Object Calisthenics	Pratique Cas concrets et choix du bon pattern	Évaluation Code review, nouveaux besoins, bilan

JOUR 1

POURQUOI LE CLEAN CODE ?

Lisibilité

Le code est beaucoup plus souvent lu qu'écrit
L'intention doit être visible rapidement
Un nom clair vaut mieux qu'un commentaire compensatoire

Maintenance

Comprendre rapidement où intervenir
Limiter les effets de bord
Pouvoir modifier sans avoir peur de casser le reste

Évolution & équipe

Faciliter les code reviews et l'onboarding
Rendre les tests et refactorings plus simples
Réduire le coût des changements futurs

Objectif : comprendre plus vite, modifier plus sereinement

DIAGNOSTIC

```
4  function calc($a, $b, $c)
5  {
6      if ($c == 1) {
7          return $a * $b - ($a * $b * 0.1);
8      } else {
9          return $a * $b;
10     }
11 }
```

Code A

```
4 ▼ function calculateTotal(
5     float $unitPrice,
6     int $quantity,
7     bool $hasDiscount
8 ▼ ): float {
9     $total = $unitPrice * $quantity;
10
11 ▼     if ($hasDiscount) {
12         return $total * 0.9;
13     }
14
15     return $total;
16 }
```

Code B

**Lequel préférez-vous maintenir dans
6 mois ? Pourquoi ?**

CLEAN CODE : CE QUE L'ON CHERCHE

Facile à changer

Les responsabilités sont claires et les dépendances maîtrisées.

Prévisible

Le comportement est local, les effets de bord sont limités.

Compréhensible

L'intention se voit sans avoir à « décoder » chaque ligne.

3 QUESTIONS À SE POSER EN LISANT DU CODE

1. Est-ce que je comprends ce que fait ce code en moins de 30 secondes ?
2. Est-ce que je sais où intervenir si le besoin métier change ?
3. Est-ce que je peux le tester ou le modifier sans devoir tout comprendre ?

5 RÉFLEXES CLEAN CODE

- Nommer pour exprimer l'intention : variables, méthodes et classes explicites
- Écrire des fonctions focalisées sur une seule intention
- Réduire la duplication plutôt que copier-coller puis oublier
- Limiter l'imbrication : early return, conditions simples, garde-fous
- Supprimer les valeurs magiques, le code mort et les abstractions inutiles

NAMING : L'INTENTION AVANT LA BRIÈVETÉ

- Un bon nom réduit le nombre d'informations que le lecteur doit garder en tête.
- À éviter : `$d`, `$x`, `calc()`, `processData()` lorsqu'on peut nommer précisément le concept métier.
- Préférer : `$daysSinceLastLogin`, `calculateOrderTotal()`, `isEligibleForDiscount()`.

CONVENTIONS PHP : PARLER LE MÊME LANGAGE

Variables et méthodes : camelCase

Classes et interfaces : PascalCase

Constantes : UPPER_SNAKE_CASE

Booléens : préfixes is / has / can

Collections : nom au pluriel

Méthodes : verbe + intention métier

Une seule langue dans le projet : français ou anglais

Respecter les conventions existantes avant d'en créer de nouvelles

L'objectif n'est pas d'avoir “le nom parfait”, mais un langage commun que toute l'équipe comprend.

TP N°1

CODE SMELLS : LES SIGNAUX D'ALERTE

Signaux dans le code qui laissent penser qu'il y a un problème de conception ou de maintenabilité. Ce n'est pas forcément un bug, mais cela mérite une analyse.

STRUCTURE

Long Method
Large Class
Long Parameter List

→ Trop de
responsabilités au
même endroit

LISIBILITÉ

Magic Values
Noms vagues
Nested Conditions
Dead Code

→ Le lecteur doit
deviner l'intention

COUPLAGE

Duplicate Code
Feature Envy
Primitive Obsession

→ Le changement se
propage trop
facilement

FONCTIONS : UNE INTENTION, UN NIVEAU D'ABSTRACTION

Une petite fonction n'est pas automatiquement une bonne fonction.

Cherchez surtout :

- une responsabilité claire ;
- un nom qui explique l'intention ;
- peu de paramètres ;
- des conditions simples ;
- des early returns lorsque cela clarifie le flux ;
- pas de booléens mystérieux qui changent complètement le comportement.

REFACTORING : CHANGER LA STRUCTURE, PAS LE COMPORTEMENT

Refactorer par petites étapes, idéalement avec des tests : le comportement observable reste identique.

Rename : rendre l'intention explicite

Extract Method : isoler une intention dans une fonction

Extract Class : séparer des responsabilités différentes

Introduce Parameter Object : réduire les listes de paramètres

Move Method : rapprocher le comportement des données qu'il utilise

Replace Conditional with Polymorphism : quand les branches représentent de vrais comportements

CONDITIONS : RÉDUIRE L'IMBRICATION

Problème : plus il y a de niveaux de if / else, plus le lecteur doit garder de contexte en tête.

Garde-fous : traiter les cas impossibles au début, puis sortir.

Early return : clarifier le chemin principal plutôt que tout enfermer dans un bloc.

Extraire les conditions complexes dans une variable ou une méthode explicite.

Objectif : rendre le scénario nominal lisible en premier.

TP N°2

DUPLICATION : LE COÛT DU COPIER-COLLER

Dupliquer du code semble rapide, mais chaque copie devient une dette.

À surveiller :

- mêmes calculs répétés ;
- mêmes conditions métier à plusieurs endroits ;
- mêmes chaînes ou statuts en dur ;
- mêmes blocs avec de petites variations.

Refactorer ne veut pas dire tout abstraire : on regroupe seulement ce qui représente réellement la même intention.

COMMENTAIRES : EXPLIQUER LE POURQUOI

- Éviter les commentaires qui répètent le code
- Renommer ou extraire plutôt que commenter pour compenser
- Commenter une décision métier, une contrainte ou un compromis
- Supprimer les commentaires obsolètes ou trompeurs

Objectif : un commentaire doit apporter une information absente du code

VALEURS MAGIQUES ET CONSTANTES

Une valeur magique est une valeur dont le sens n'est pas explicite à la lecture.

À ÉVITER

```
if ($total > 100)  
    $total * 0.9  
$status === 'P'
```

→ Le lecteur doit deviner la règle métier

À PRÉFÉRER

```
FREE_SHIPPING_THRESHOLD  
DISCOUNT_RATE  
STATUS_PENDING
```

→ Le nom porte l'intention

QUESTION

Cette valeur est-elle
une règle métier ?
Va-t-elle changer ?
Est-elle utilisée ailleurs ?

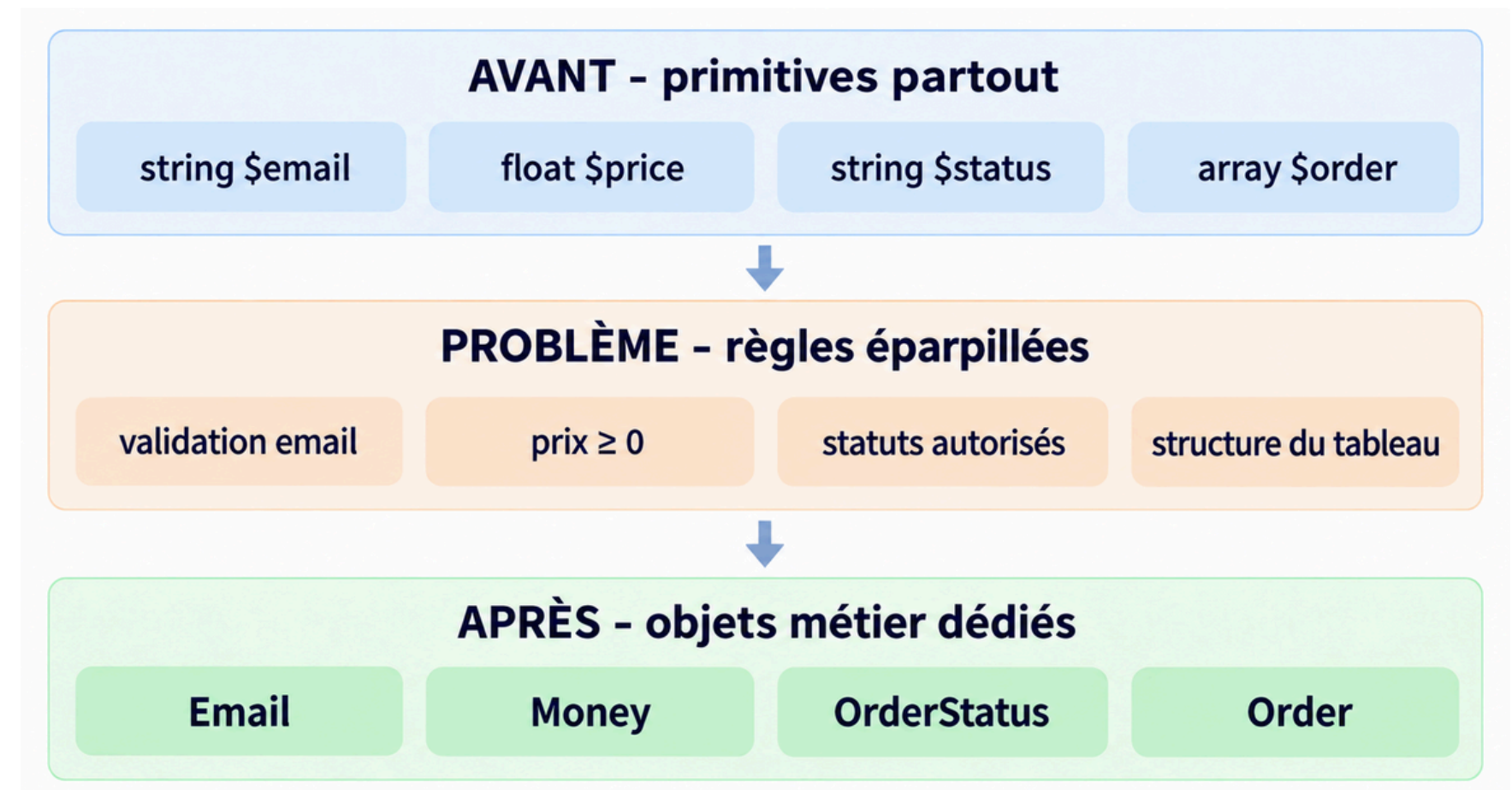
→ Si oui, nommez-la.

PRIMITIVE OBSESSION

Tout représenter avec des string, int, float ou array rend le code fragile.

Exemples :

- un prix représenté par float ;
- une adresse email représentée par string ;
- un statut représenté par string ;
- une commande représentée par array.



Quand un concept porte une règle métier, il mérite souvent un objet dédié.

SÉPARER LES RESPONSABILITÉS

Une classe qui fait trop de choses devient difficile à comprendre et à modifier.

Exemple de classe trop large :

- valider une commande ;
- calculer le total ;
- appliquer une remise ;
- sauvegarder en base ;
- envoyer un email ;
- générer une facture.

Question clé : combien de raisons différentes cette classe a-t-elle de changer ?

TP N°3

LE FIL ROUGE PHP

On part d'une petite application de commande volontairement imparfaite.

MODÈLE

Customer
Product
Order
OrderItem

→ Les données métier principales

SERVICE

OrderService
PaymentService
Mailer
InvoiceGenerator

→ Beaucoup trop de logique au départ

OBJECTIF

Lire
Critiquer
Refactorer
Faire évoluer

→ Sans changer le comportement attendu

LECTURE DE CODE EN GROUPE

Avant de refactorer, on commence par comprendre.

Méthode :

- identifier les entrées et sorties ;
- repérer les règles métier ;
- nommer les responsabilités ;
- lister les zones risquées ;
- formuler une intention de refactoring.

Un bon refactoring commence souvent par une bonne lecture du code.

TP N°4

JOUR 2

SOLID : POURQUOI ?

Problèmes concrets

Une classe change trop souvent

Une fonctionnalité impose de modifier partout

Une dépendance technique est difficile à remplacer

Des tests sont difficiles à écrire



SOLID

5 principes de conception

SRP

OCP

LSP

ISP

DIP



Ce que l'on cherche

Responsabilités mieux séparées

Code plus évolutif

Couplage réduit

Code plus testable

SRP : SINGLE RESPONSIBILITY PRINCIPLE

Une classe doit avoir une responsabilité principale et une seule raison claire de changer.

Exemples

OrderValidator : vérifier qu'une commande est valide

OrderCalculator : calculer total, remises et frais

OrderRepository : gérer la persistance

OrderMailer : envoyer les notifications

InvoiceGenerator : produire la facture

Question : si une règle change, quelle classe doit changer ?

TP N°5

OCP EN PRATIQUE (OPEN / CLOSED PRINCIPLE)

Ajouter un nouveau comportement en créant une nouvelle classe plutôt qu'en modifiant la classe centrale.

Sans OCP

À chaque nouveau moyen de paiement,
il faut modifier la classe existante.

```
class PaymentService
{
    public function pay(string $method, float $amount)
    {
        if ($method === 'stripe') {
            // logique Stripe
        } elseif ($method === 'paypal') {
            // logique PayPal
        } elseif ($method === 'virement') {
            // logique virement bancaire
        } elseif ($method === 'sur_place') {
            // logique paiement sur place
        }
    }
}
```

stripe

PayPal

Virement
bancaire

Sur place

✗ Le code devient fragile et difficile à maintenir.



Avec OCP,
on ajoute un
nouveau moyen
de paiement en
créant une nouvelle
classe.

Avec OCP

Chaque moyen de paiement est une classe.
On peut en ajouter un nouveau sans modifier l'existant.

```
interface PaymentMethod
{
    public function pay(float $amount): void;
}
```

```
class StripePayment
{
    public function pay(float $amount)
    {
        // logique Stripe
    }
}
```

stripe

```
class PayPalPayment
{
    public function pay(float $amount)
    {
        // logique PayPal
    }
}
```

PayPal

```
class BankTransferPayment
{
    public function pay(float $amount)
    {
        // logique virement
    }
}
```

Virement
bancaire

```
class OnSitePayment
{
    public function pay(float $amount)
    {
        // logique sur place
    }
}
```

Sur place



Le code est plus modulaire, plus évolutif et plus facile à tester.

TP N°6

DIP EN PRATIQUE (DEPENDENCY INVERSION PRINCIPLE)

Une classe métier ne devrait pas dépendre directement d'un outil technique



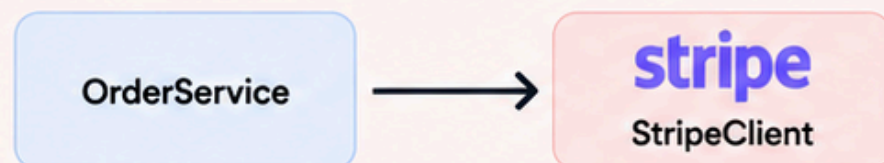
À éviter

Dépendance directe à un outil technique

```
class OrderService
{
    private StripeClient $stripe;

    public function __construct()
    {
        $this->stripe = new StripeClient('sk_...');
    }

    public function payer(float $montant): void
    {
        $this->stripe->charge($montant);
    }
}
```



Le code métier est directement lié à un outil technique.
Difficile à remplacer et à tester.



On dépend
d'une abstraction
et non d'une
implémentation
concrète.



À préférer

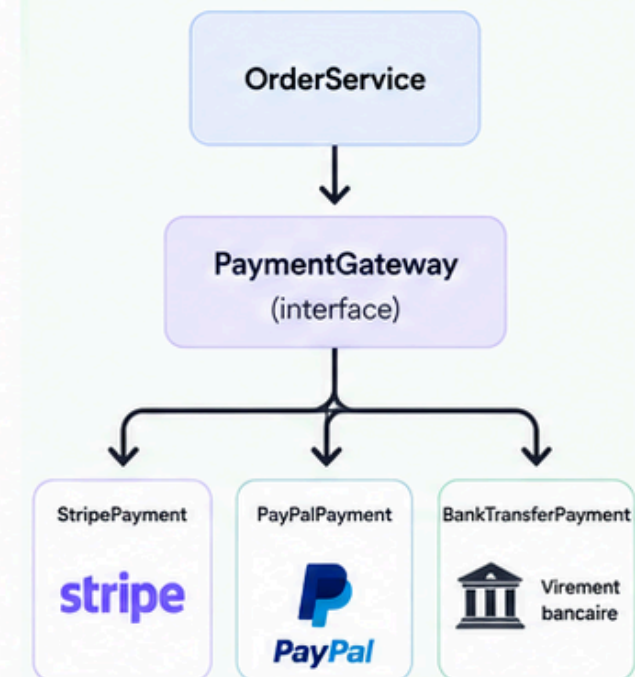
Dépendance à une interface (abstraction)

```
interface PaymentGateway
{
    public function payer(float $montant): void;
}

class OrderService
{
    private PaymentGateway $paymentGateway;

    public function __construct(PaymentGateway $paymentGateway)
    {
        $this->paymentGateway = $paymentGateway;
    }

    public function payer(float $montant): void
    {
        $this->paymentGateway->payer($montant);
    }
}
```



Avantages

- Remplacement plus simple (changement de prestataire)
- Tests plus simples (on peut utiliser un mock)
- Code métier moins lié à l'infrastructure

TP N°7

LSP : LISKOV SUBSTITUTION PRINCIPLE

Un objet d'une classe fille doit pouvoir remplacer un objet de la classe parente sans casser le comportement attendu.

Signal d'alerte :

- méthode héritée mais non applicable ;
- exception “not supported” ;
- comportement contraire à ce que promet le type parent.

En pratique : attention aux hiérarchies forcées. La composition est souvent plus simple que l'héritage.

ISP EN PRATIQUE (INTERFACE SEGREGATION PRINCIPLE)

Une classe ne devrait pas être obligée d'implémenter des méthodes qu'elle n'utilise pas.

À éviter :

interface trop large avec payer(), rembourser(), imprimer(), notifier(), archiver().

À préférer :

- Payable ;
- Refundable ;
- Printable ;
- Notifiable.

De petites interfaces rendent les dépendances plus précises et les tests plus simples.

TP N°8

JOUR 3

OBJECT CALISTHENICS : CONTRAINTES UTILES

Les Object Calisthenics sont des règles volontairement contraignantes pour pousser à écrire du code plus simple. Ce ne sont pas des lois absolues : ce sont des exercices pour progresser → **Code plus propre, moins de complexité, plus facile à faire évoluer.**

1

Une seule indentation par méthode

Chaque méthode doit tenir sur un seul niveau d'indentation.

✖ Avant

✔ Après

```
public function total() {  
  if ($this->items) {  
    foreach ($this->items as $item) {  
      return $item->price;  
    }  
  }  
}
```

```
public function total() {  
  return $this->calculateTotalFromItems();  
}
```

Encourage la décomposition en petites méthodes.

2

Pas de else

Utiliser des retours anticipés (early return) pour éviter les else.

✖ Avant

✔ Après

```
public function canBook() {  
  if ($this->isFull()) {  
    return false;  
  } else {  
    return true;  
  }  
}
```

```
public function canBook() {  
  if ($this->isFull()) {  
    return false;  
  }  
  return true;  
}
```

Du code plus lisible et plus simple à comprendre.

3

Pas de switch

Utiliser le polymorphisme au lieu des switch.

✖ Avant

✔ Après

```
public function getPrice() {  
  switch ($this->type) {  
    case 'vip':  
      return 80;  
    case 'standard':  
      return 100;  
    default:  
      return 120;  
  }  
}
```

```
interface Ticket {  
  public function getPrice(): int;  
}  
  
class VipTicket implements Ticket {  
  public function getPrice(): int {  
    return 80;  
  }  
}
```

Le comportement est porté par les objets eux-mêmes.

4

Un seul point dans une méthode

Une méthode ne doit appeler qu'une seule autre méthode.

✖ Avant

✔ Après

```
public function process() {  
  $this->validate();  
  $this->calculate();  
  $this->save();  
}
```

```
public function process() {  
  $this->validateAndProcess();  
}
```

```
private function validateAndProcess() {  
  $this->validate();  
  $this->calculate();  
  $this->save();  
}
```

Favorise des objets plus petits et plus cohérents.

5

Pas d'abréviations

Des noms clairs et explicites.

✖ Avant

✔ Après

```
public function calcTot() {  
  // ...  
}
```

```
public function calculateTotal() {  
  // ...  
}
```

Un code qui se lit comme une histoire.

6

Moins de 3 variables d'instance

Limiter le nombre d'attributs par classe.

✖ Avant

✔ Après

```
class Order {  
  private $id;  
  private $customer;  
  private $items;  
  private $status;  
  private $createdAt;  
}
```

```
class Order {  
  private $id;  
  private $customer;  
  private $items;  
  // Status et dates gérés  
  // par d'autres objets  
}
```

Des objets plus simples et plus focalisés.

7

Pas plus de 3 paramètres

Limiter le nombre de paramètres par méthode.

✖ Avant

✔ Après

```
public function create(  
  string $name,  
  string $email,  
  string $type,  
  string $source  
) {}
```

```
class CustomerData {  
  public function __construct(  
    public string $name,  
    public string $email,  
    public string $type,  
    public string $source  
  ) {}  
}  
  
public function create(  
  CustomerData $data  
) {}
```

Encourage des objets de valeur et plus de cohésion.

8

Pas de getter / setter

Éviter d'exposer l'état interne. Préférer des comportements métier.

✖ Avant

✔ Après

```
class Order {  
  private $status;  
  
  public function getStatus() {  
    return $this->status;  
  }  
  
  public function setStatus($status) {  
    $this->status = $status;  
  }  
}
```

```
class Order {  
  private $status;  
  
  public function markAsPaid() {  
    $this->status = 'paid';  
  }  
  
  public function isPaid(): bool {  
    return $this->status === 'paid';  
  }  
}
```

On manipule l'objet via son métier, pas ses données.

9

Toujours du premier niveau d'abstraction

Éviter de mélanger les niveaux de détail dans une même méthode.

✖ Avant

✔ Après

```
public function checkout() {  
  $total = 0;  
  foreach ($this->items as $i) {  
    $total += $i->price;  
  }  
  $this->charge($total);  
}
```

```
public function checkout() {  
  $total = $this->calculateTotal();  
  $this->charge($total);  
}
```

Chaque méthode parle du même niveau métier.

10

Pas de classes avec plus de 50 lignes

Une classe doit rester petite et focalisée.

✖ Avant

✔ Après

```
// 240 lignes  
class OrderService {  
  // ...  
}
```

```
class OrderProcessor {}  
class OrderNotifier {}  
class OrderRepository {}  
// etc.
```

Si une classe grossit, on la découpe.

UN PATTERN N'EST PAS UNE RECETTE MAGIQUE

Un design pattern est une solution réutilisable à un problème de conception fréquent.

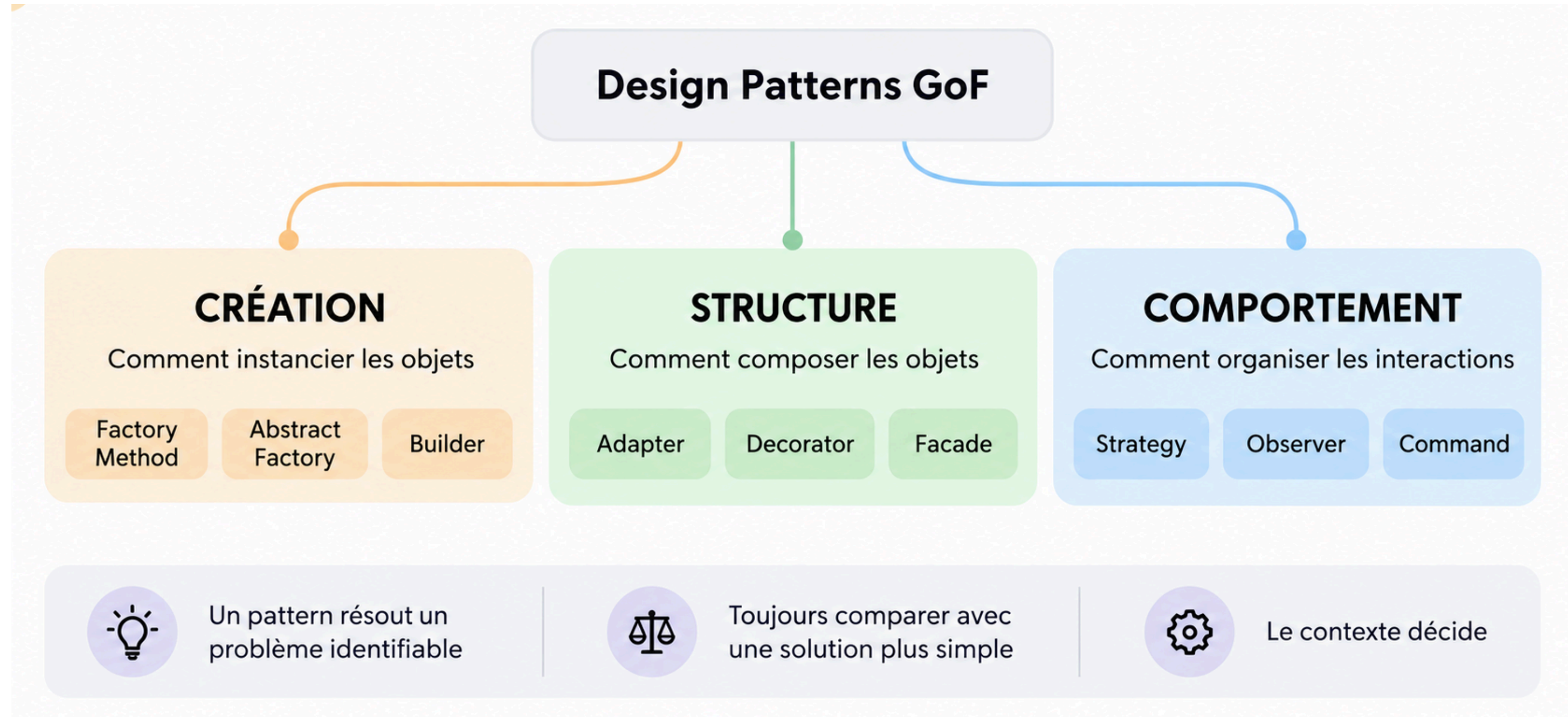
Il ne faut pas partir du pattern pour chercher où le placer.

Il faut partir du problème :

- duplication de comportements ;
- objet incompatible avec une interface attendue ;
- actions à déclencher après un événement ;
- création d'objets qui devient complexe.

Ensuite seulement, on choisit éventuellement un pattern.

FAMILLES DE DESIGN PATTERNS GOF



STRATEGY

Strategy permet de rendre interchangeable un comportement



Avant

Plusieurs if dans une classe

```
class OrderService
{
    public function process(float $amount, string $mode)
    {
        if ($mode === 'carte') {
            // logique de paiement par carte
        } elseif ($mode === 'paypal') {
            // logique de paiement PayPal
        } elseif ($mode === 'virement') {
            // logique de virement bancaire
        } else {
            throw new InvalidArgumentException('Mode inconnu');
        }
    }
}
```



- La classe grossit à chaque nouveau mode.
- Difficile à maintenir et à tester.
- On modifie la classe existante.

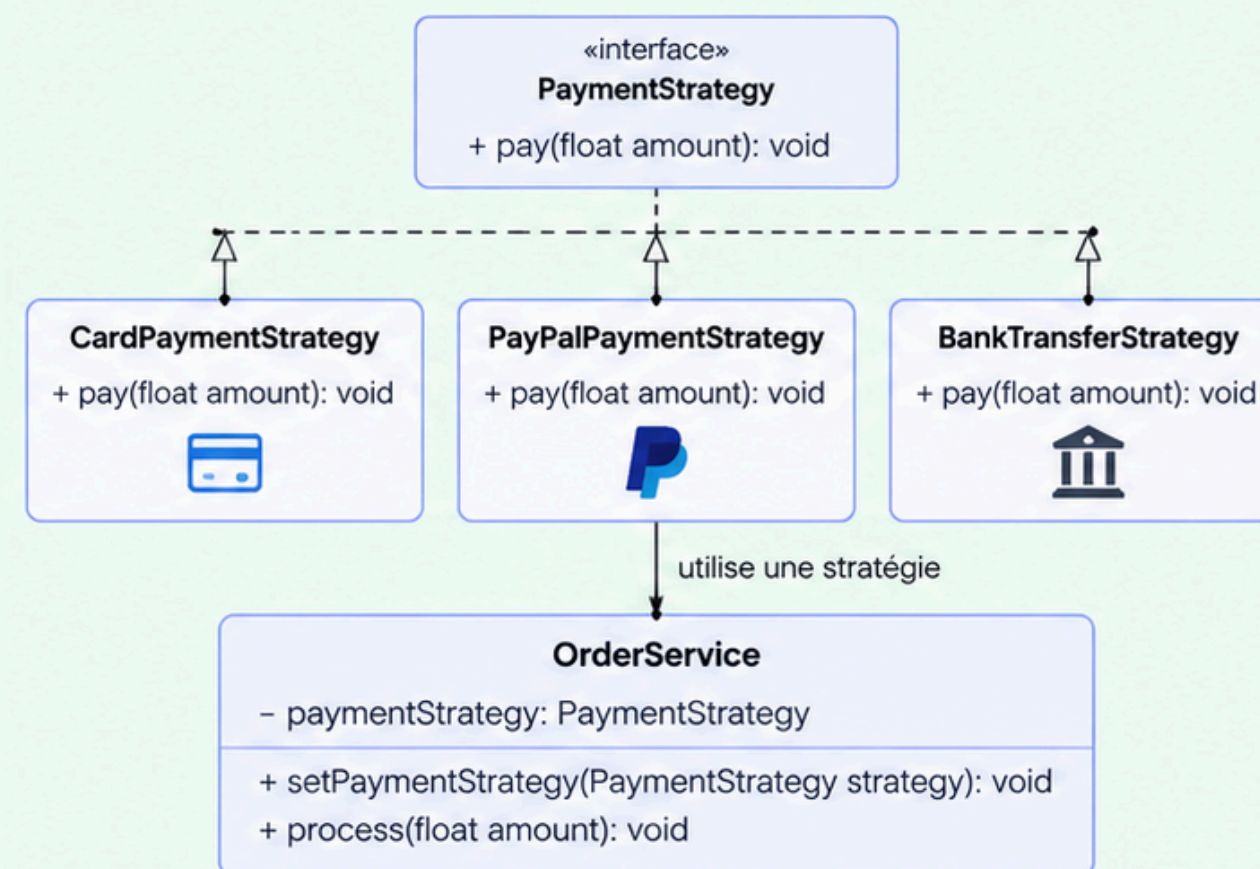


On extrait
le comportement
dans une interface
et on le rend
interchangeable.



Avec Strategy

Chaque stratégie implémente la même interface

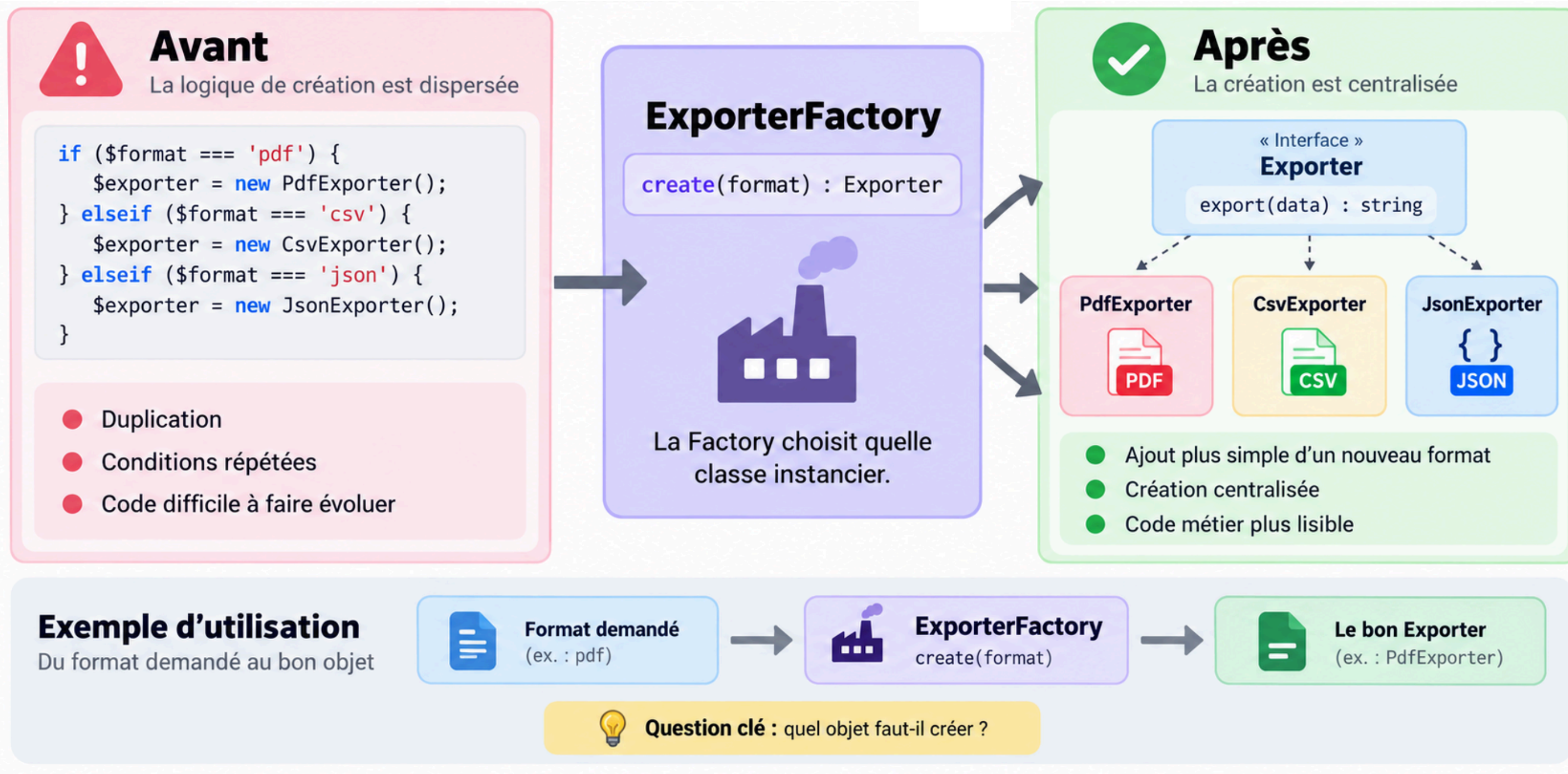


Exemples d'utilisation

- mode de paiement (carte, PayPal, virement...)
- calcul de livraison (standard, express...)
- règle de remise (aucune, fidélité, promo...)
- export de données (PDF, CSV, JSON...)

SIMPLE FACTORY

Centraliser la création des objets



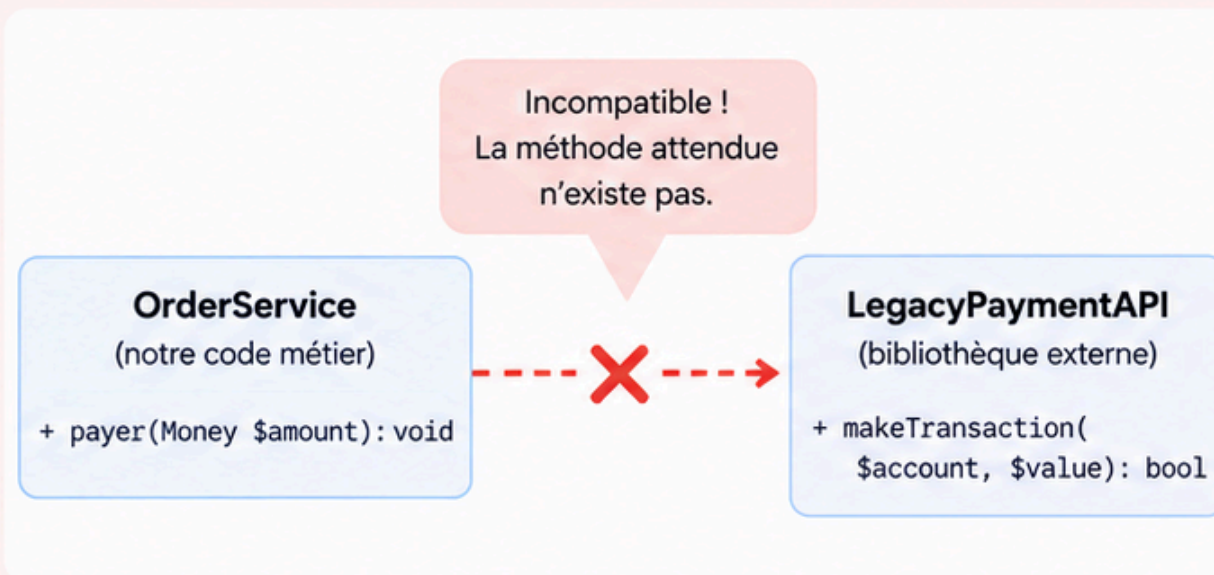
ADAPTER

Adapter permet de rendre compatible une classe existante avec l'interface attendue par notre code



Avant

Interfaces incompatibles

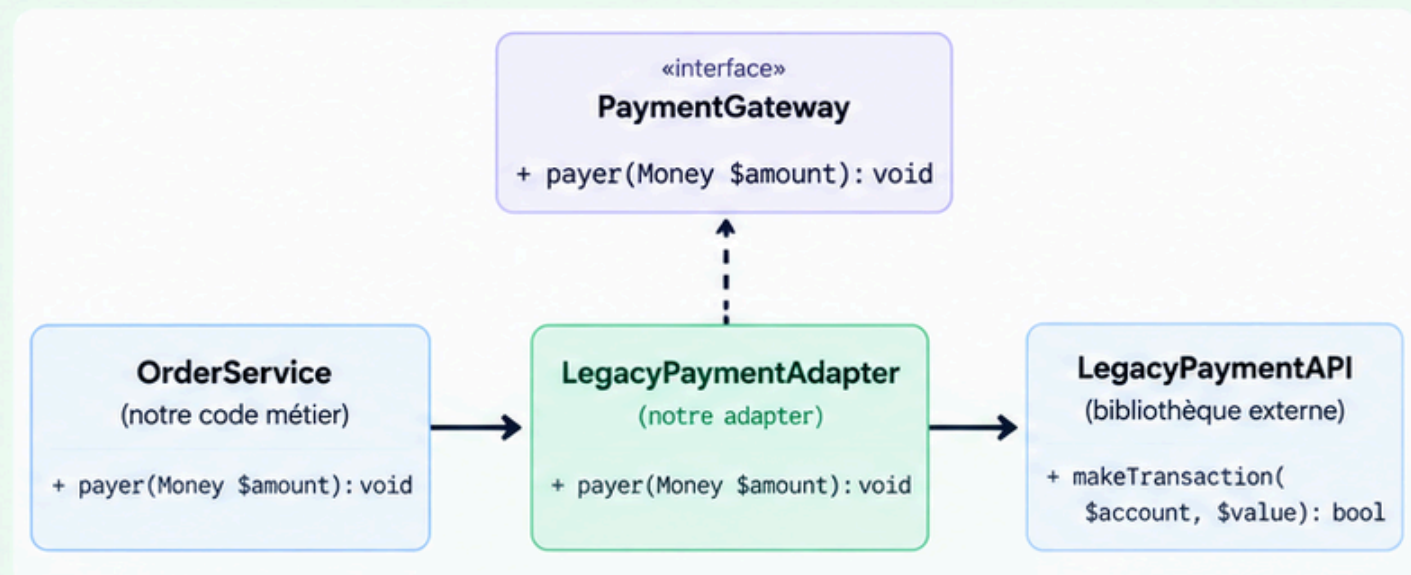


- Interfaces différentes
- Notre code ne peut pas utiliser directement la bibliothèque
- On devrait modifier notre code métier (à éviter)



Avec Adapter

Une couche d'adaptation



- L'adapter traduit notre appel vers l'API existante
- Notre code métier reste dépendant de l'interface PaymentGateway
- La bibliothèque externe reste isolée



Idée clé : Adapter joue le rôle de traducteur entre notre interface et une API incompatible.

OBSERVER

Observer permet de déclencher plusieurs réactions lorsqu'un événement se produit



Avant

Le service connaît toutes les réactions

```
class OrderService
{
    public function validateOrder(Order $order): void
    {
        // Valider la commande

        // Envoyer un email
        $this->emailService->send($order);

        // Décrémenter le stock
        $this->stockService->decrement($order);

        // Ajouter des points de fidélité
        $this->loyaltyService->addPoints($order);

        // Envoyer un événement analytics
        $this->analyticsService->track($order);
    }
}
```



- Le service principal connaît toutes les réactions.
- Code plus difficile à maintenir.
- À chaque nouvelle réaction, il faut modifier la classe.

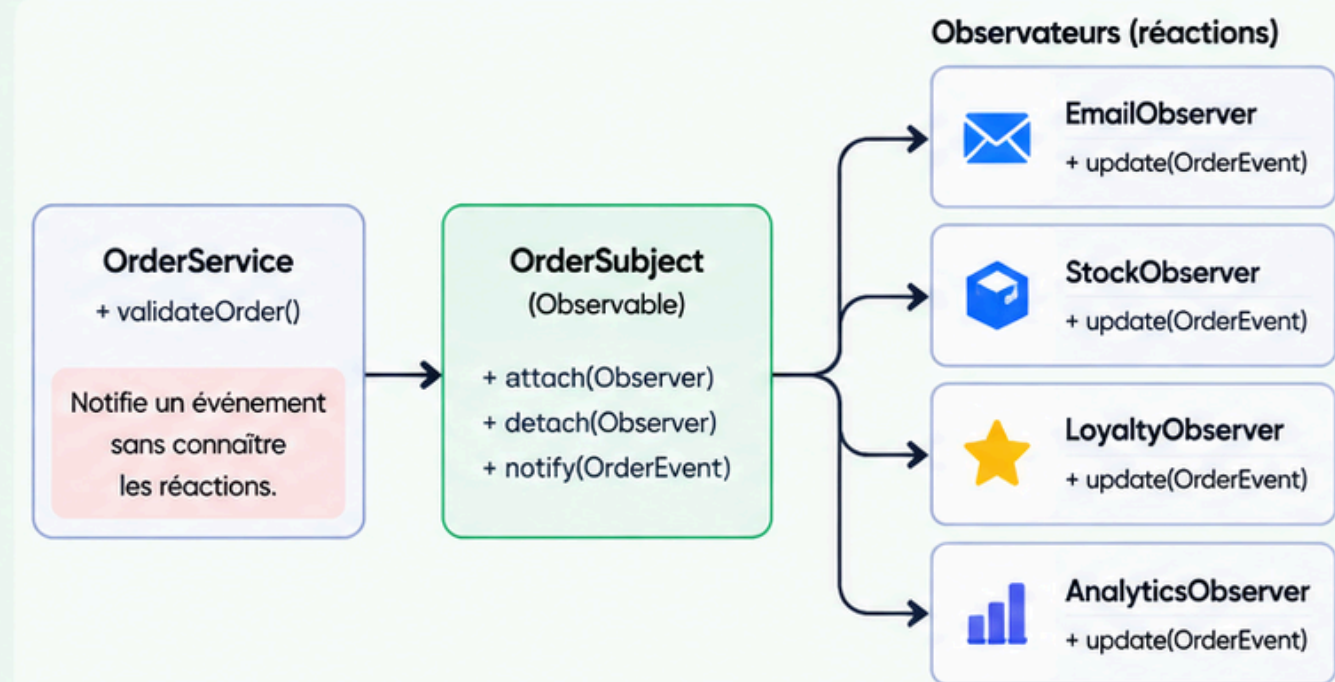


Observer
déconnecte
l'émetteur
des réactions.



Avec Observer

Le service notifie, les observateurs réagissent



- Le service principal n'a pas besoin de connaître les détails.
- On peut ajouter/retirer des réactions facilement.
- Code plus modulaire et plus maintenable.



Idée clé : Observer définit une relation 1 → N entre un émetteur d'événement et plusieurs observateurs.

DECORATOR

Decorator permet d'ajouter un comportement autour d'un objet sans modifier sa classe



Avant

Plusieurs responsabilités dans la même classe

```
class PaymentService
{
    public function pay(float $amount): void
    {
        // Vérification
        $this->validate($amount);

        // Log
        $this->logger->info("Paielement de {$amount}");

        // Mesure du temps
        $start = microtime(true);
        $this->gateway->pay($amount);
        $duration = microtime(true) - $start;
        $this->logger->info("Durée : {$duration}s");
    }
}
```



- La classe grossit rapidement.
- Difficile à maintenir et à faire évoluer.
- On ne peut pas ajouter/retirer un comportement facilement.

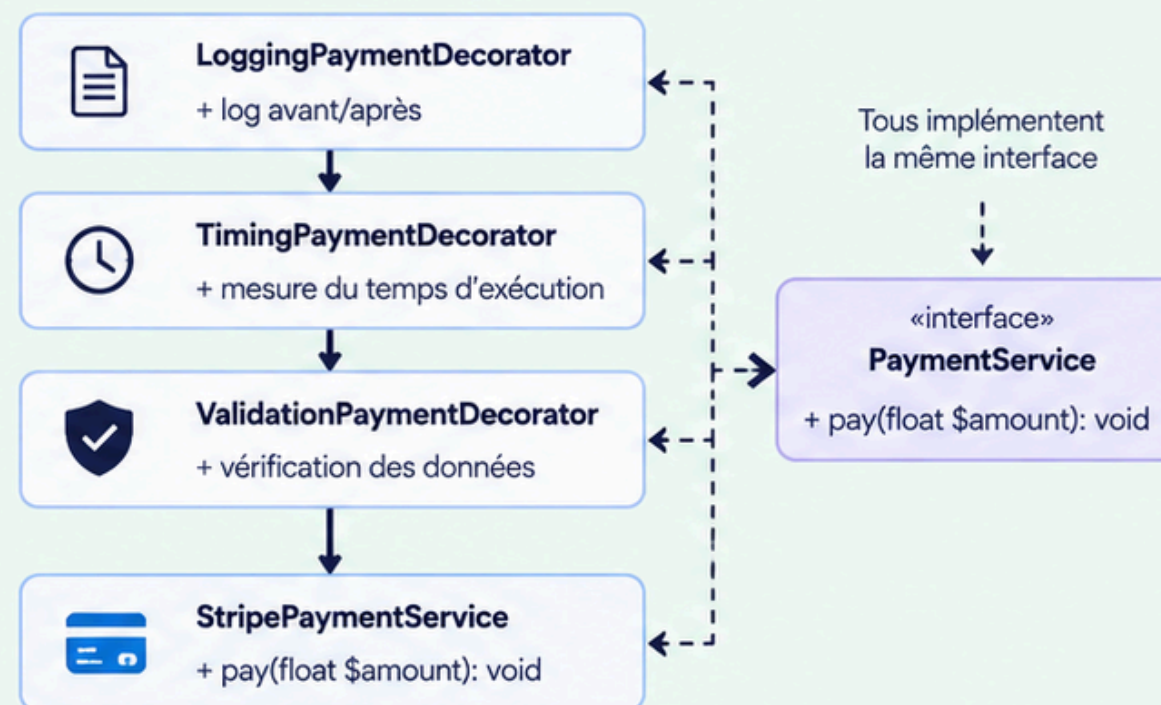


On ajoute des comportements de manière modulaire autour de l'objet.



Avec Decorator

Des comportements ajoutés dynamiquement



- On peut ajouter/retirer des comportements facilement.
- Le décorateur respecte la même interface que l'objet décoré.
- Code plus modulaire et plus maintenable.



Idée clé : Decorator permet d'ajouter des responsabilités sans modifier la classe d'origine, en les "empilant" autour de l'objet.

CHOISIR UN PATTERN

Questions à se poser avant d'utiliser un pattern :

- Quel problème concret suis-je en train de résoudre ?
- Est-ce que le code actuel change souvent ?
- Est-ce que plusieurs comportements doivent devenir interchangeables ?
- Est-ce qu'une dépendance externe doit être isolée ?
- Est-ce qu'une solution plus simple suffit ?

Un pattern est pertinent s'il réduit la complexité globale, pas s'il la déplace.

PATTERN OU OVERENGINEERING ?

Un design pattern peut être utile, mais il peut aussi rendre un petit problème inutilement complexe.

Signaux d'overengineering :

- beaucoup de classes pour un besoin très simple ;
- abstraction créée avant d'avoir plusieurs cas ;
- vocabulaire technique qui cache le métier ;
- code plus difficile à lire après refactoring.

Règle simple : choisir la solution la plus claire qui répond au besoin actuel.

TP N°10